

Parte 1

Ejercicio 1.1

Recordar que se estableció como correcto $\theta = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix}$

$$\rightarrow h_{\theta}(x_i^{(i)}) = \theta_0 + \theta_1 x_i^{(i)}$$

↓
0.2

300 valores candidatos para θ_1 :

Puede ser: $\theta_{1\text{grid}} = \{-1, \dots, -0.5, \dots, 0, \dots, 0.5, \dots, 1.5, \dots, 2\}$

Intervalo que se grafica
como *zemple*

$$\left. \begin{aligned} J_1 &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + (-1)x_i^{(i)}) - y_i^{(i)}]^2 \\ J_2 &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + (-0.9)x_i^{(i)}) - y_i^{(i)}]^2 \\ &\vdots \\ J_{300} &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + 2x_i^{(i)}) - y_i^{(i)}]^2 \end{aligned} \right\}$$

$\theta_1 = 0.5$ minimiza J .

La superficie de coste: Con 2 parámetros θ_1, θ_2 , J es una superficie en $\mathbb{R}^3$

Hasta ahora J_{grid} dependía de un solo parámetro θ_1 (con θ_0 fijo, bariado en una dimensión).

Pero la superficie de coste real depende de 2 parámetros θ_0 y θ_1 .

Se necesita hacer un barrido sobre ambos. Se debe evaluar J en todas las combinaciones posibles de θ_0 y θ_1 .

→ `np.linspace(-0.6, 1.0, 80)` : Genera 80 candidatos de θ_0

→ `np.linspace(-0.6, 1.0, 80)` : Genera 80 candidatos de θ_1

$80 \times 80 = 6.400$ combinaciones

→ `meshgrid` : devuelve 2 arreglos a la vez

→ Recorrer las 6400 combinaciones (θ_0, θ_1) que armé meshgrid
calcula J para cada una y guarda todos esos 6,400
resultados en J_g .

$$J_g = \begin{bmatrix} J_g(1,1) & \dots & J_g(1,80) \\ J_g(2,1) & & \vdots \\ \vdots & & \vdots \\ \vdots & & \vdots \end{bmatrix}_{80 \times 80}$$

Parte 2: Gradiente Descendente

Para bajar por la superficie se usa la regla de actualización

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Caso 1D:

Ensayar con función mucho más simple: Una variable

$$f(w) = (w-1)^2 \quad ; \quad f_{\min} \text{ en } w=1$$
$$f'(w) = 2(w-1)$$

Aplicar regla $w := w - \alpha f'(w)$ → w valores que hacen que
 $h(w)$ (en este caso) se
ajuste mejor a los datos reales.
minimizando J

- α pequeño: converge, pero gusta muchas iteraciones
- α adecuado: baja rápido y se queda en el mín.
- α muy grande: sobrepasa el mínimo en cada paso y diverge.

Ejercicio 2.1: Implementar el gradiente y el descenso

En este caso, se debe calcular:

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

Parte 3: Regresión Multivariada

versión vectorizada

Ya no es una única característica, si no n .

De este modo

$$X = \begin{bmatrix} x_1^{(1)} \\ x_1^{(2)} \\ \vdots \\ x_1^{(m)} \end{bmatrix} \quad \longrightarrow \quad X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & & x_n^{(2)} \\ \vdots & \vdots & & \vdots \\ 1 & x_1^{(m)} & & x_n^{(m)} \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

$$h = X\theta \quad ; \quad J(\theta) = \frac{1}{2m} (h-y)^T (h-y) \quad ; \quad \nabla_{\theta} J = \frac{1}{m} X^T (h-y)$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (X\theta - y)$$

• def agregar_sesgo: X_f : es un array

if $X_f.ndim = 1$: $X_f = X_f.reshape(-1, 1)$

Saber cuantas dimensiones tiene el array.

Si tiene una sola característica $\rightarrow$ lo guarda como $[\#, \#, \dots \#]$
 $\text{ndim} = 1$.

reshape (-1, 1)

↓ Número de
no calcula las Columnas
filas necesarias

Sin importar si se le da 1 ó n características, X_f siempre termina siendo una matriz y no un vector plano.

- def h_lin: $x_b @ \Theta_{\text{lin}}$ multiplication matrix vector

$\downarrow$
 si $X_b = (4 \times 4)$ 4 parámetros
 4 células, 4 columnas $\theta_0, \theta_1, \theta_2, \theta_3$

$$h_{\theta}(x) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1+0 \\ 3+0 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} h_{\theta}(x^{(1)}) \\ h_{\theta}(x^{(2)}) \end{bmatrix}$$

En general : $h_0(x) = \begin{bmatrix} h_0(x^{(1)}) \\ \vdots \\ h_0(x^{(m)}) \end{bmatrix}$

- J_{lin} : Calcula vector de residuos

$$J_{\text{in}}: \begin{bmatrix} J_1 \\ \vdots \\ J_m \end{bmatrix}$$

Ejercicio 3.1: El gradiente vectorizado

Traducir $\nabla_{\theta} J = \frac{1}{m} X^T (X\theta - y)$ a una sola línea de numpy.

$$= \frac{1}{m} X.T (h_{\theta}(x) - y)$$

• `def_descenso_vect`: Al igual que en el caso de 1 característica

`Theta = None` → se pone `None` porque no sabemos cuantos parámetros va a tener θ . Depende del # de características.
Esto añade un valor vacío

`Theta`: